

Lecture 9: Simple Linear Regression

PSTAT100: Data Science — Concepts and Analysis

John Inston 

johninston@ucsb.edu

University of California, Santa Barbara

April 25, 2026

Overview

Aims of the lecture

- Understand **regression** as estimation of the conditional expectation $\mathbb{E}[Y \mid X]$.
- Specify the **simple linear regression (SLR) model** and the Gauss-Markov assumptions.
- Derive the **ordinary least squares (OLS)** estimators analytically.
- Understand the **properties** of OLS: unbiasedness, sampling variance, and BLUE.
- Conduct **inference** — t -tests and confidence intervals — on regression coefficients.
- Measure **goodness of fit** using the SST/SSR/SSE decomposition and R^2 .
- Distinguish **confidence intervals** from **prediction intervals** for new data.

Required Libraries

In this lecture we will be using the following libraries:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from scipy.stats import t as t_dist
```

Figure Styles

```
1 sns.set_style('whitegrid')
2 sns.set_palette('Set2')
```

Introduction to Linear Regression

What is Regression?

Connecting to what we know

- In **Lecture 7** we introduced the conditional expectation $\mathbb{E}[Y \mid X = x]$: the expected value of Y when X is fixed at x .
- This is the **regression function** — the theoretically optimal prediction of Y given X .

Regression

Regression is the task of estimating the conditional expectation function $\mu(x) = \mathbb{E}[Y \mid X = x]$ from observed data $(x_1, y_1), \dots, (x_n, y_n)$.

Why impose linearity?

- The true function $\mu(x)$ could take any shape, but we cannot estimate it without assumptions.
- **Linear regression** restricts the model to $\mu(x) = \beta_0 + \beta_1 x$ — a straight line.
- This is a **modelling choice**: it is interpretable, tractable with small n , and often a reasonable local approximation.

Types of Regression

Simple vs. multiple

- **Simple linear regression (SLR)**: one predictor X . This lecture.
- **Multiple linear regression (MLR)**: p predictors $X_1, \dots, X_p$. Lecture 11.

Linear vs. nonlinear

- “Linear” refers to linearity in the **parameters** β , not necessarily in X itself.
 - $Y = \beta_0 + \beta_1 X^2$ is still a linear model (we treat $X' = X^2$ as the predictor).
 - $Y = \beta_0 \exp(\beta_1 X)$ is nonlinear in β and requires different methods.
- Much of machine learning can be viewed as flexible, nonlinear regression.

Regression in the supervised learning framework

Task	Response Y	Example
Regression	Continuous	House price, temperature, body mass
Classification	Categorical	Spam / not-spam, tumour type

The Simple Linear Regression Model

The Statistical Model

Simple Linear Regression Model

We assume the data-generating process is:

$$Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad i = 1, \dots, n,$$

where:

- x_i is the **observed** predictor value (treated as fixed / non-random).
- Y_i is the **response** (random).
- β_0 is the **intercept** — the expected value of Y when $x = 0$.
- β_1 is the **slope** — the expected change in Y per one-unit increase in x .
- ε_i is the **error** — the deviation of Y_i from the line; captures all unexplained variability.

Two components of the model

The Gauss-Markov Assumptions

Four conditions for classical linear regression

1. **Linearity**: $\mathbb{E}[\varepsilon_i] = 0$ for all i .
 - The true conditional mean is a linear function of x .
2. **Independence**: $\varepsilon_1, \dots, \varepsilon_n$ are mutually independent.
 - Errors from different observations do not influence one another.
3. **Homoscedasticity**: $\text{Var}(\varepsilon_i) = \sigma^2$ for all i .
 - The spread of errors is constant across all values of x .
4. **Normality**: $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$.
 - Errors are normally distributed. Required for exact inference with small n ; asymptotically justified by the CLT for large n .

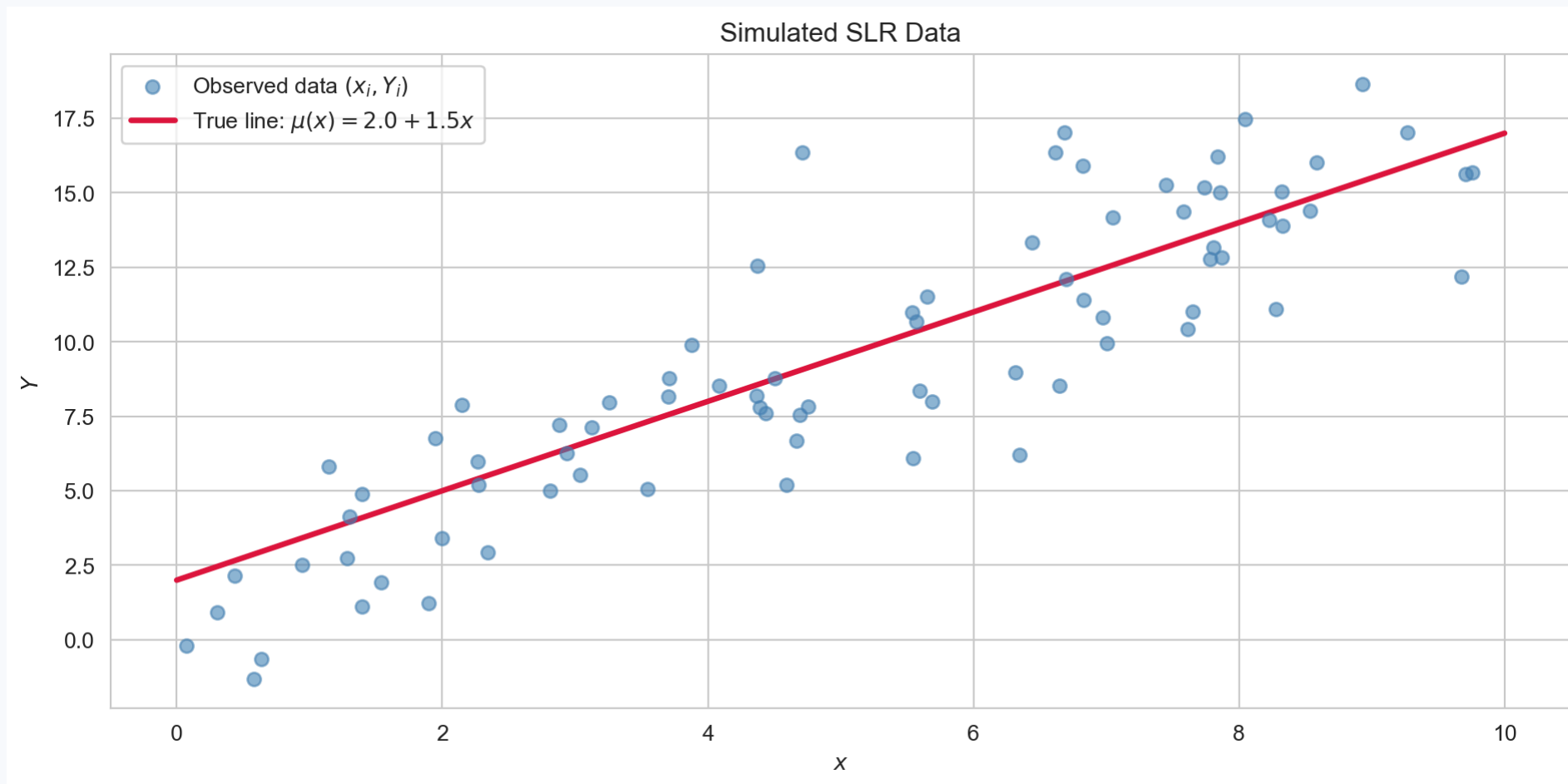
Together, 1-4 imply that

$$Y_i \stackrel{\text{iid}}{\sim} \mathcal{N}(\beta_0 + \beta_1 x_i, \sigma^2).$$

Visualising the SLR Model

```
1 rng = np.random.default_rng(42)
2 n, beta0_true, beta1_true, sigma_true = 80, 2.0, 1.5, 2.5
3 x = rng.uniform(0, 10, size=n)
4 y = beta0_true + beta1_true * x + rng.normal(0, sigma_true, size=n)
5
6 x_line = np.linspace(0, 10, 200)
7
8 fig, ax = plt.subplots(figsize=(10, 5))
9 ax.scatter(x, y, alpha=0.6, color='steelblue', label='Observed data  $(x_i, Y_i)$ ', zorder=3)
10 ax.plot(x_line, beta0_true + beta1_true * x_line, color='crimson', lw=2.5,
11         label=rf'True line:  $\mu(x) = \{beta0\_true\} + \{beta1\_true\}x$ ')
12 ax.set_xlabel('$x$')
13 ax.set_ylabel('$Y$')
14 ax.set_title('Simulated SLR Data')
15 ax.legend()
16 plt.tight_layout()
17 plt.show()
```

Visualising the SLR Model

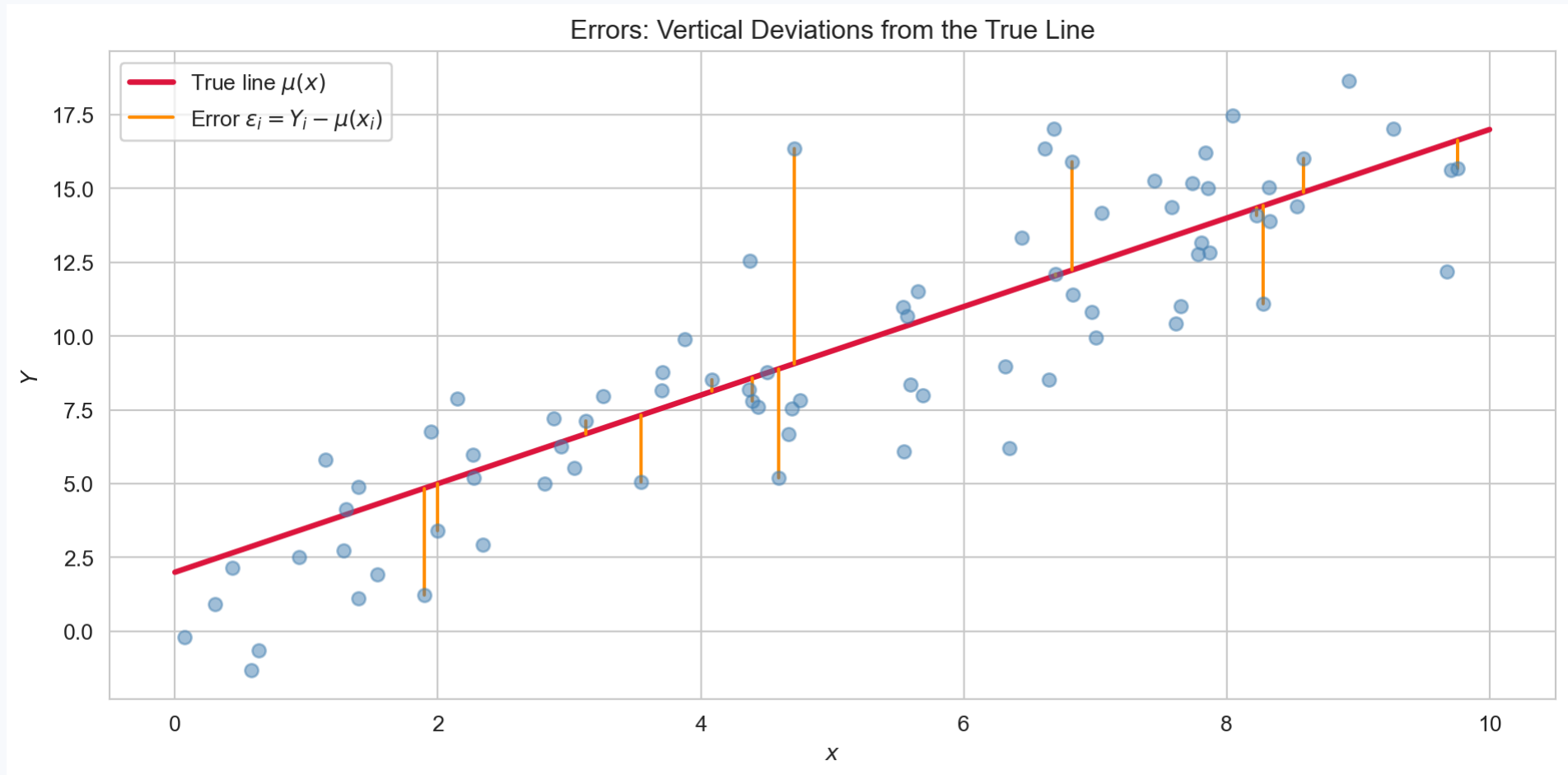


Simulated data from $Y = 2 + 1.5x + \varepsilon$ ($\varepsilon \sim N(0, 2.5^2)$). The true line is shown in red.

Visualising the Error Term

```
1 fig, ax = plt.subplots(figsize=(10, 5))
2 ax.scatter(x, y, alpha=0.5, color='steelblue', zorder=3)
3 ax.plot(x_line, beta0_true + beta1_true * x_line, color='crimson', lw=2.5,
4         label='True line  $\mu(x)$ ')
5
6 # Show errors for a selection of points
7 rng_idx = np.random.default_rng(0)
8 show_idx = rng_idx.choice(n, size=14, replace=False)
9 for i in show_idx:
10     mu_i = beta0_true + beta1_true * x[i]
11     ax.plot([x[i], x[i]], [mu_i, y[i]], color='darkorange', lw=1.5, zorder=2)
12
13 ax.plot(
14     [], [],
15     color='darkorange',
16     lw=1.5,
17     label='Error  $\epsilon_i = Y_i - \mu(x_i)$ '
18 )
19 ax.set_xlabel('$x$')
20 ax.set_ylabel('$Y$')
21 ax.set_title('Errors: Vertical Deviations from the True Line')
22 ax.legend()
23 plt.tight_layout()
24 plt.show()
```

Visualising the Error Term



Errors ε_i are the vertical deviations of each observation from the true line.

Ordinary Least Squares Estimation

The OLS Criterion

Choosing the best-fitting line

- In practice β_0 and β_1 are **unknown** and must be **estimated** from data.
- For any candidate line $\hat{y} = b_0 + b_1x$, the **residual** for observation i is:

$$e_i = y_i - (b_0 + b_1x_i).$$

Ordinary Least Squares (OLS)

The **OLS estimators** $\hat{\beta}_0$ and $\hat{\beta}_1$ minimise the **residual sum of squares**:

$$\text{RSS}(b_0, b_1) = \sum_{i=1}^n (y_i - b_0 - b_1x_i)^2.$$

- Squaring makes the criterion **differentiable** and penalises large deviations heavily.
- It yields **closed-form** estimators — we can solve **analytically** by setting derivatives to zero.

Deriving the OLS Estimators

Setting partial derivatives to zero

Taking $\partial \text{RSS} / \partial b_0 = 0$ and $\partial \text{RSS} / \partial b_1 = 0$:

$$-2 \sum_{i=1}^n (y_i - b_0 - b_1 x_i) = 0, \quad \& \quad -2 \sum_{i=1}^n x_i (y_i - b_0 - b_1 x_i) = 0.$$

Solving these **normal equations** gives:

$$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x},$$

where $S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2$ and $S_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$.

Key insights

- The OLS line always passes through the **sample means** $(\bar{x}, \bar{y})$.
- $\hat{\beta}_1$ is proportional to the **sample covariance** of x and y .

Relationship to MLE under Normality

If we assume GM1-GM4 then

$$Y_i \sim \mathcal{N}(\beta_0 + \beta_1 x_i, \sigma^2), \quad i = 1, \dots, n.$$

The **likelihood function** is:

$$L(\beta_0, \beta_1, \sigma^2) = \prod_{i=1}^n f_Y(y_i) = (2\pi\sigma^2)^{-\frac{n}{2}} \exp \left(-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \right).$$

The **log-likelihood function** is:

$$l(\beta_0, \beta_1, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \underbrace{\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2}_{\text{RSS}}.$$

Maximizing L with respect to β_0 and β_1 is **equivalent** to minimizing the RSS, so:

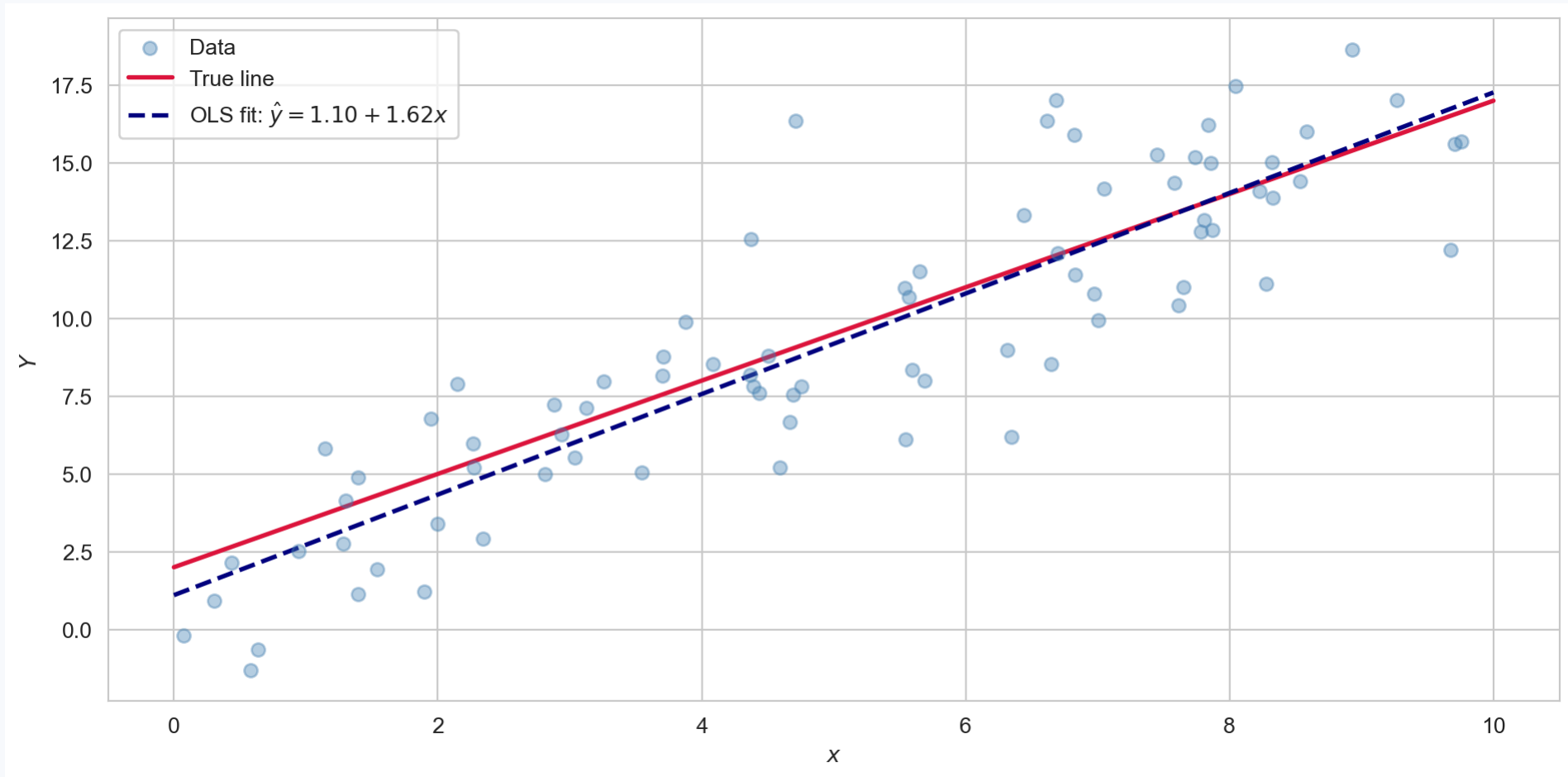
$$\hat{\beta}_0^{\text{MLE}} = \hat{\beta}_0^{\text{OLS}}, \quad \hat{\beta}_1^{\text{MLE}} = \hat{\beta}_1^{\text{OLS}}.$$

Computing OLS from Scratch

```
1 # Step 1: Compute the sufficient statistics
2 x_bar = x.mean()
3 y_bar = y.mean()
4 S_xx = np.sum((x - x_bar)**2)
5 S_xy = np.sum((x - x_bar) * (y - y_bar))
6
7 # Step 2: OLS slope and intercept
8 beta1_hat = S_xy / S_xx
9 beta0_hat = y_bar - beta1_hat * x_bar
10
11 print(f"True:       $\beta_0 = \{{\text{beta0\_true:.2f}}\}, \beta_1 = \{{\text{beta1\_true:.2f}}\}")
12 print(f"Estimated:  $\hat{\beta}_0 = \{{\text{beta0\_hat:.3f}}\}, \hat{\beta}_1 = \{{\text{beta1\_hat:.3f}}\}")
13
14 # Step 3: Plot
15 fig, ax = plt.subplots(figsize=(10, 5))
16 ax.scatter(x, y, alpha=0.4, color='steelblue', label='Data', zorder=3)
17 ax.plot(x_line, beta0_true + beta1_true * x_line, 'crimson', lw=2, label='True line')
18 ax.plot(x_line, beta0_hat + beta1_hat * x_line, 'navy', lw=2, linestyle='--',
19        label=f'OLS fit:  $\hat{y} = \{{\text{beta0\_hat:.2f}}\} + \{{\text{beta1\_hat:.2f}}\}x')$ ')
20 ax.set_xlabel('$x$'); ax.set_ylabel('$Y$')
21 ax.legend(); plt.tight_layout(); plt.show()$$ 
```

Computing OLS from Scratch

True: $\beta_0 = 2.00$, $\beta_1 = 1.50$
Estimated: $\hat{\beta}_0 = 1.103$, $\hat{\beta}_1 = 1.616$



OLS estimated line (navy dashed) vs. the true line (red). With $n=80$ observations the estimate is very close.

Fitted Values and Residuals

! Fitted Values and Residuals

The **fitted value** for observation i is $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$.

The **residual** is the difference between the observed and fitted response:

$$e_i = y_i - \hat{y}_i.$$

- The residuals satisfy the following properties:
 - $\sum_{i=1}^n e_i = 0$: residuals sum to zero.
 - $\sum_{i=1}^n x_i e_i = 0$: residuals are uncorrelated with x .
 - $\sum_{i=1}^n \hat{y}_i e_i = 0$: residuals are uncorrelated with fitted values.
- These three identities follow directly from the normal equations.

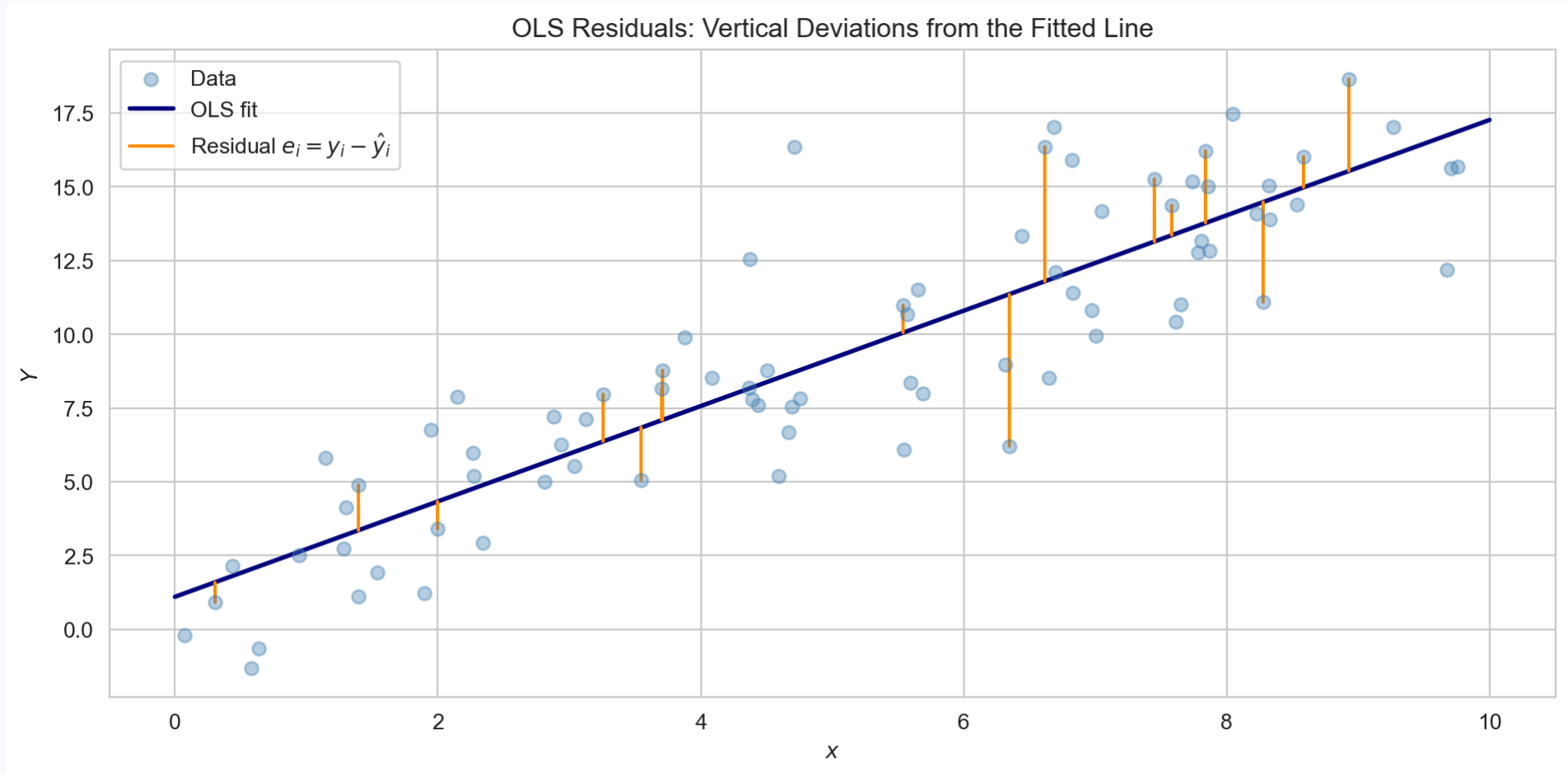
```
1 y_hat = beta0_hat + beta1_hat * x
2 residuals = y - y_hat
3 print(f"Σ e_i = {residuals.sum():.2e}")
4 print(f"Σ x_i e_i = {(x * residuals).sum():.2e}")
5 print(f"Σ ŷ_i e_i = {(y_hat * residuals).sum():.2e}")
```

```
Σ e_i = -3.55e-14
Σ x_i e_i = -1.99e-13
Σ ŷ_i e_i = -4.55e-13
```

Visualising Residuals

```
1 fig, ax = plt.subplots(figsize=(10, 5))
2 ax.scatter(x, y, alpha=0.4, color='steelblue', zorder=3, label='Data')
3 ax.plot(x_line, beta0_hat + beta1_hat * x_line, 'navy', lw=2, label='OLS fit')
4
5 rng_idx2 = np.random.default_rng(1)
6 show_idx2 = rng_idx2.choice(n, size=16, replace=False)
7 for i in show_idx2:
8     ax.plot([x[i], x[i]], [y_hat[i], y[i]], color='darkorange', lw=1.5, zorder=2)
9
10 ax.plot([], [], color='darkorange', lw=1.5, label='Residual  $e_i = y_i - \hat{y}_i$ ')
11 ax.set_xlabel('$x$'); ax.set_ylabel('$Y$')
12 ax.set_title('OLS Residuals: Vertical Deviations from the Fitted Line')
13 ax.legend()
14 plt.tight_layout()
15 plt.show()
```

Visualising Residuals



Residuals e_i are the vertical distances from each data point to the OLS line.

Properties of OLS Estimators

✓ Unbiasedness

OLS recovers the true parameters on average

We can write $\hat{\beta}_1 = \sum_{i=1}^n c_i Y_i$ where $c_i = (x_i - \bar{x}) / S_{xx}$ (linear combination).

Since $Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$ and $\mathbb{E}[\varepsilon_i] = 0$:

$$\mathbb{E}[\hat{\beta}_1] = \sum_{i=1}^n c_i \mathbb{E}[Y_i] = \beta_0 \underbrace{\sum_{i=1}^n c_i}_{=\frac{n\bar{x}-n\bar{x}}{S_{xx}}=0} + \beta_1 \underbrace{\sum_{i=1}^n c_i x_i}_{=\frac{S_{xx}}{S_{xx}}=1} = \beta_1. \quad \checkmark$$

An analogous argument gives $\mathbb{E}[\hat{\beta}_0] = \beta_0$.

- Both estimators are **unbiased** under GM1–GM3. **Normality (GM4) is not needed.**
- On average across many datasets, OLS will return the true parameter values.

Sampling Variance of OLS Estimators

Under GM1–GM4, the exact sampling variances are:

$$\text{Var}(\hat{\beta}_1) = \frac{\sigma^2}{S_{xx}}, \quad \text{Var}(\hat{\beta}_0) = \sigma^2 \left(\frac{1}{n} + \frac{\bar{x}^2}{S_{xx}} \right).$$

What drives precision?

Factor	Effect on $\text{Var}(\hat{\beta}_1)$
More spread in x ($\uparrow S_{xx}$)	$\downarrow$ Variance — better leverage
Larger sample size ($\uparrow n$)	$\downarrow$ Variance — more information
Less noise ($\downarrow \sigma^2$)	$\downarrow$ Variance — cleaner signal

Estimating σ^2

Since σ^2 is unknown, we estimate it with the **residual variance**:

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n - 2}.$$

We divide by $n - 2$ (not n) because estimating $\hat{\beta}_0$ and $\hat{\beta}_1$ costs **two degrees of freedom**.

The Gauss-Markov Theorem

! Gauss-Markov Theorem

Under assumptions GM1–GM3 (linearity, independence, homoscedasticity — **normality is not required**), the OLS estimators $\hat{\beta}_0$ and $\hat{\beta}_1$ are **BLUE**:

Best **L**inear **U**nbiased **E**stimators.

Unpacking “BLUE”

- **Linear**: the estimator is a linear function of the responses $Y_1, \dots, Y_n$.
- **Unbiased**: $\mathbb{E}[\hat{\beta}_j] = \beta_j$ (as shown above).
- **Best**: among *all* linear unbiased estimators, OLS has the **smallest variance**.

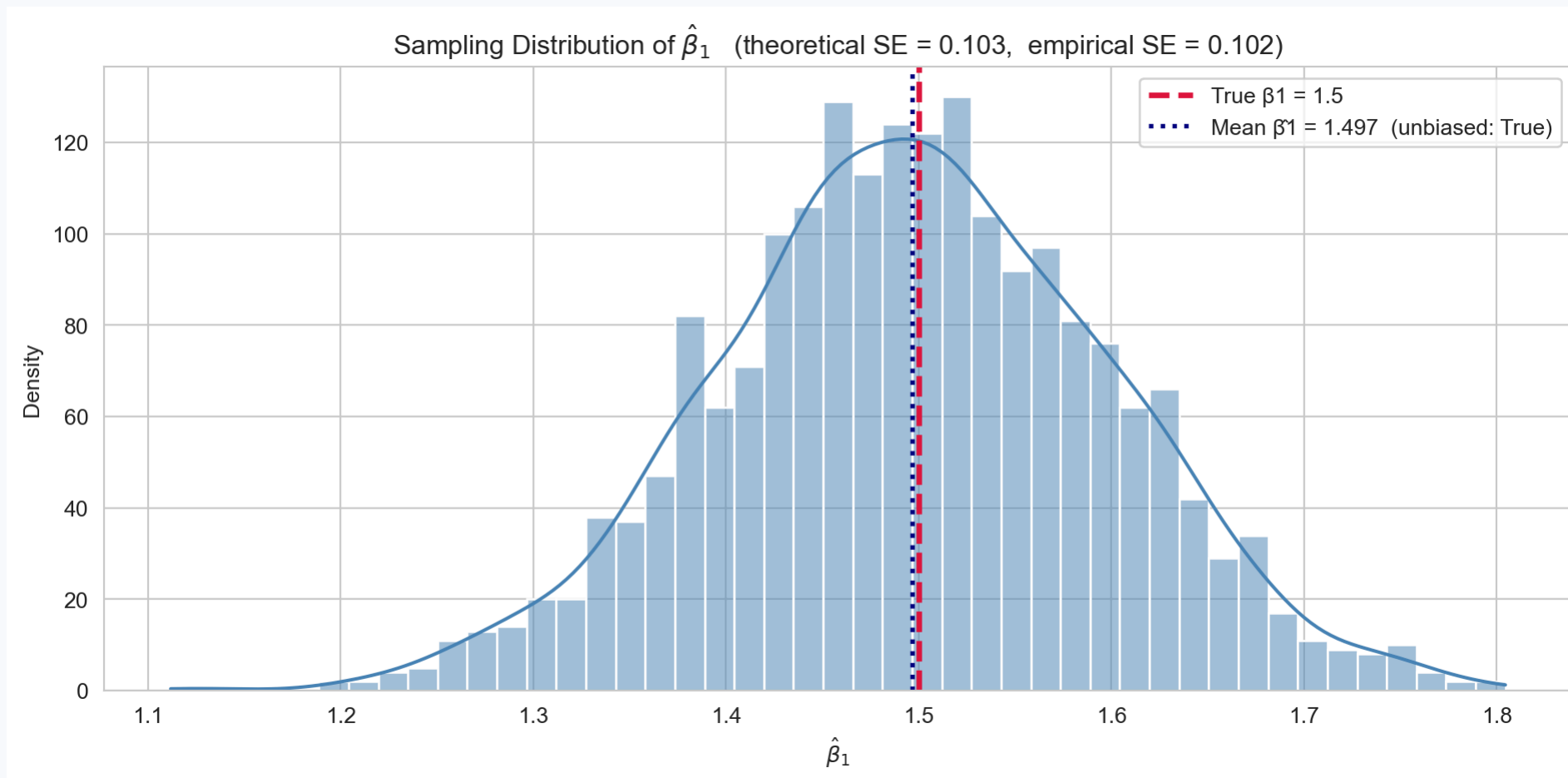
The practical message

- We don't need normality for OLS to be the optimal linear estimator.
- Normality is only needed for **exact** t -based inference in small samples.
- For large n , the CLT justifies normal-based inference regardless.

Simulating OLS Properties

```
1 rng2 = np.random.default_rng(7)
2 beta1_sims = []
3 for _ in range(2000):
4     y_sim = beta0_true + beta1_true * x + rng2.normal(0, sigma_true, size=n)
5     S_xy_s = np.sum((x - x_bar) * (y_sim - y_sim.mean()))
6     beta1_sims.append(S_xy_s / S_xx)
7
8 beta1_sims = np.array(beta1_sims)
9 theoretical_se = sigma_true / np.sqrt(S_xx)
10
11 fig, ax = plt.subplots(figsize=(10, 5))
12 sns.histplot(beta1_sims, bins=45, kde=True, color='steelblue', ax=ax)
13 ax.axvline(beta1_true, color='crimson', lw=2.5, linestyle='--',
14             label=f'True  $\beta_1 = \{beta1\_true\}$ ')
15 ax.axvline(beta1_sims.mean(), color='navy', lw=2, linestyle=':',
16             label=f'Mean  $\hat{\beta}_1 = \{beta1\_sims.mean():.3f\}$  (unbiased: True)')
17 ax.set_xlabel(r'$\hat{\beta}_1$'); ax.set_ylabel('Density')
18 ax.set_title(f'Sampling Distribution of  $\hat{\beta}_1$  '
19             f'(theoretical SE = {theoretical_se:.3f}, empirical SE = {beta1_sims.std():.3f})')
20 ax.legend()
21 plt.tight_layout()
22 plt.show()
```

Simulating OLS Properties



Sampling distribution of $\hat{\beta}_1$ across 2 000 simulated datasets. The estimator is centred on the true value $\beta_1 = 1.5$.

Statistical Inference for Regression

The Sampling Distribution of $\hat{\beta}_1$

Under normality (GM4)

Since $\hat{\beta}_1 = \sum_i c_i Y_i$ is a linear combination of independent normals:

$$\hat{\beta}_1 \sim \mathcal{N}\left(\beta_1, \frac{\sigma^2}{S_{xx}}\right).$$

Replacing σ with sample standard deviation s_x

Because we estimate σ^2 from the same data, we incur extra uncertainty. This leads to:

$$T = \frac{\hat{\beta}_1 - \beta_1}{s_{\hat{\beta}_1}} \sim t_{n-2}, \quad s_{\hat{\beta}_1} = \frac{s_x}{\sqrt{S_{xx}}}.$$

- The degrees of freedom are $n - 2$ because we estimated **two** parameters (β_0 and β_1).
- An analogous result holds for $\hat{\beta}_0$ with its own standard error.

t -Tests for Regression Coefficients

Testing whether a predictor has a linear effect

The most common test is:

$$H_0 : \beta_1 = 0 \quad \text{vs.} \quad H_1 : \beta_1 \neq 0.$$

If $\beta_1 = 0$, then X has **no linear association** with Y in the population.

Test statistic and p -value

$$T = \frac{\hat{\beta}_1}{s_{\hat{\beta}_1}} \sim t_{n-2} \quad \text{under } H_0.$$

$$p\text{-value} = 2\mathbb{P}(t_{n-2} \geq |T_{\text{obs}}|).$$

```
1 RSS = np.sum(residuals**2)
2 sigma_hat = np.sqrt(RSS / (n - 2))
3 SE_b1 = sigma_hat / np.sqrt(S_xx)
4 SE_b0 = sigma_hat * np.sqrt(1/n + x_bar**2 / S_xx)
5
6 t_b1 = beta1_hat / SE_b1
7 p_b1 = 2 * t_dist.sf(abs(t_b1), df=n - 2)
8 print(f"β1 = {beta1_hat:.4f}, SE = {SE_b1:.4f}")
9 print(f"t = {t_b1:.3f}, p = {p_b1:.2e}")
```

```
β1 = 1.6164, SE = 0.0968
t = 16.691, p = 1.84e-27
```

Confidence Intervals for the Coefficients

A $(1 - \alpha) \times 100\%$ confidence interval for β_j is:

$$\hat{\beta}_j \pm t_{\alpha/2, n-2} \cdot s_{\hat{\beta}_j}.$$

Interpretation

- Values of β_j **outside** the CI are inconsistent with the data at the α level.
- If the CI for β_1 **excludes zero**, we reject $H_0 : \beta_1 = 0$ at the same level.
- Reporting a CI is more informative than a p -value alone — it communicates **magnitude and uncertainty**, not just significance.

```
1 alpha = 0.05
2 t_crit = t_dist.ppf(1 - alpha/2, df=n - 2)
3
4 CI_b1 = (beta1_hat - t_crit * SE_b1, beta1_hat + t_crit * SE_b1)
5 CI_b0 = (beta0_hat - t_crit * SE_b0, beta0_hat + t_crit * SE_b0)
6
7 print(f"95% CI for  $\beta_1$ : ({CI_b1[0]:.3f}, {CI_b1[1]:.3f})")
8 print(f"95% CI for  $\beta_0$ : ({CI_b0[0]:.3f}, {CI_b0[1]:.3f})")
9 print(f"True  $\beta_1$  = {beta1_true} ← inside CI? {CI_b1[0] < beta1_true < CI_b1[1]}")
```

```
95% CI for  $\beta_1$ : (1.424, 1.809)
95% CI for  $\beta_0$ : (-0.004, 2.209)
True  $\beta_1$  = 1.5 ← inside CI? True
```

Goodness of Fit

Decomposing Variation

How much of Y 's variability does X explain?

! Variance Decomposition (ANOVA Identity)

$$\underbrace{\sum_{i=1}^n (y_i - \bar{y})^2}_{\text{SST (Total)}} = \underbrace{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}_{\text{SSR (Regression)}} + \underbrace{\sum_{i=1}^n e_i^2}_{\text{SSE (Residual)}} .$$

Quantity	Name	Degrees of Freedom	What it measures
SST	Total SS	$n - 1$	Total variability in Y
SSR	Regression SS	1	Variability explained by the model
SSE	Residual SS	$n - 2$	Variability unexplained (error)

The R^2 Statistic

⚠ Coefficient of Determination R^2

$$R^2 = \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\text{SSE}}{\text{SST}}.$$

R^2 is the **proportion of variance in Y explained by the linear model**.

Properties of R^2

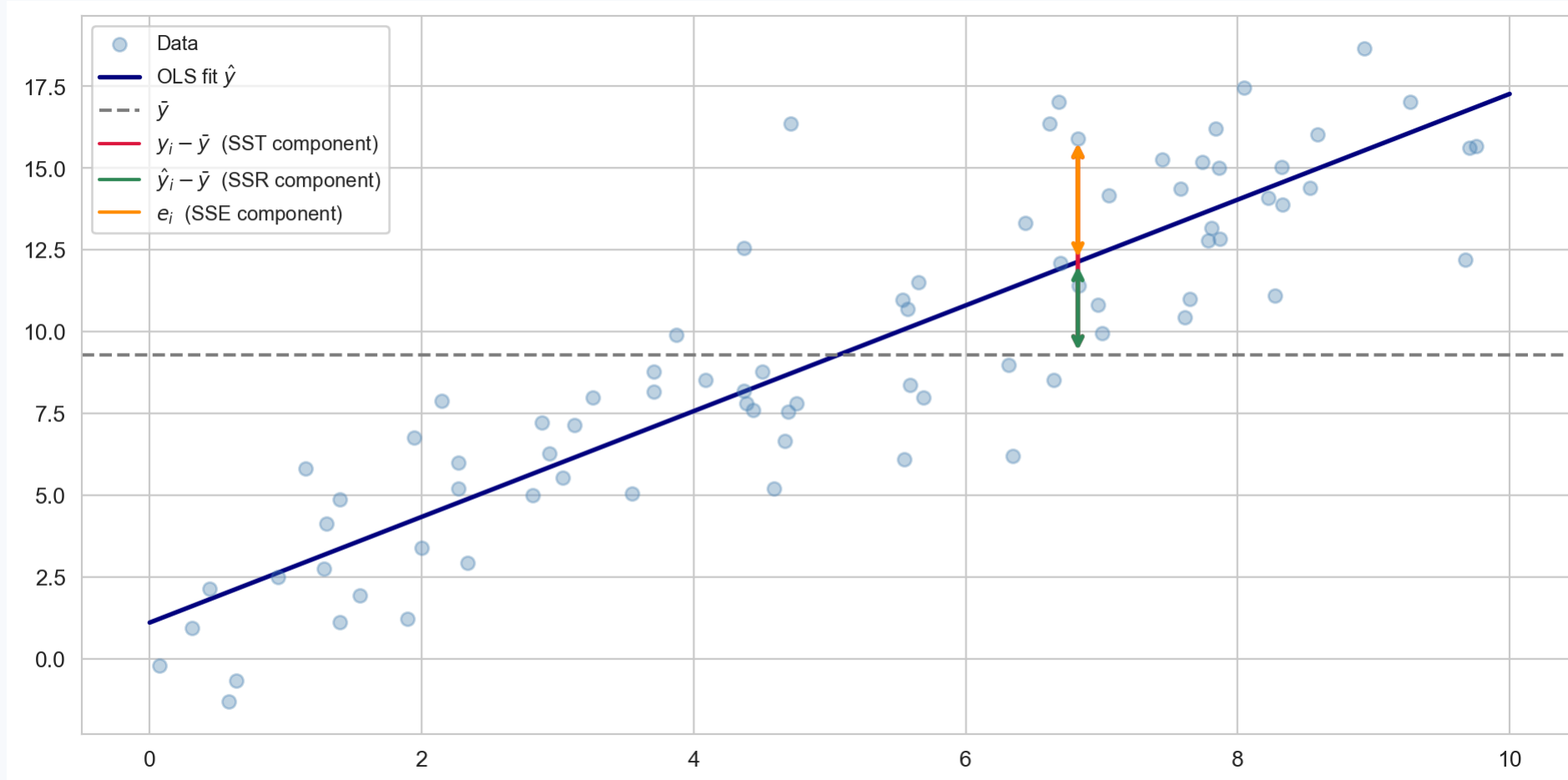
- $0 \leq R^2 \leq 1$.
- $R^2 = 0$: the model explains nothing (the flat line $\hat{y} = \bar{y}$ fits just as well).
- $R^2 = 1$: all points lie exactly on the line — perfect fit.
- For SLR only: $R^2 = r^2$, the **square of the Pearson correlation coefficient**.
- A high R^2 does **not** mean the model assumptions are satisfied.
- R^2 always increases when predictors are added — addressed with **adjusted R^2** in Lecture 11.

Computing R^2 and the Decomposition

```
1 SST = np.sum((y - y_bar)**2)
2 SSE_v = np.sum(residuals**2)
3 SSR = SST - SSE_v
4 R2 = SSR / SST
5 r = np.corrcoef(x, y)[0, 1]
6
7 print(f"SST = {SST:.2f} (total)")
8 print(f"SSR = {SSR:.2f} (explained)")
9 print(f"SSE = {SSE_v:.2f} (residual)")
10 print(f" $R^2$  = SSR/SST = {R2:.4f}")
11 print(f"r = {r:.4f} →  $r^2$  = {r**2:.4f} ✓")
12
13 fig, ax = plt.subplots(figsize=(10, 5))
14 ax.scatter(x, y, alpha=0.35, color='steelblue', zorder=3, label='Data')
15 ax.plot(x_line, beta0_hat + beta1_hat * x_line, 'navy', lw=2, label='OLS fit  $\hat{y}$ ')
16 ax.axhline(y_bar, color='gray', lw=1.5, linestyle='--', label=' $\bar{y}$ ')
17
18 i0 = np.argmin(np.abs(x - 6.8))
19 ax.annotate('', xy=(x[i0], y[i0]), xytext=(x[i0], y_bar),
20             arrowprops=dict(arrowstyle='<->', color='crimson', lw=2))
21 ax.annotate('', xy=(x[i0], y_hat[i0]), xytext=(x[i0], y_bar),
22             arrowprops=dict(arrowstyle='<->', color='seagreen', lw=2))
23 ax.annotate('', xy=(x[i0], y[i0]), xytext=(x[i0], y_hat[i0]),
24             arrowprops=dict(arrowstyle='<->', color='darkorange', lw=2))
25
26 ax.plot([], [], color='crimson', label=f' $y_i - \bar{y}$  (SST component)')
27 ax.plot([], [], color='seagreen', label=f' $\hat{y}_i - \bar{y}$  (SSR component)')
28 ax.plot([], [], color='darkorange', label=f' $e_i$  (SSE component)')
29 ax.legend(fontsize=9): plt.tight_layout(): plt.show()
```

Computing R^2 and the Decomposition

SST = 1952.34 (total)
SSR = 1525.27 (explained)
SSE = 427.07 (residual)
 $R^2 = \text{SSR}/\text{SST} = 0.7813$
 $r = 0.8839 \rightarrow r^2 = 0.7813 \quad \checkmark$



The variance decomposition for one point: SST (red) = SSR (green) + SSE (orange).

Prediction

Point Predictions

Using the fitted model for new data

For a new input x^* , the **point prediction** is:

$$\hat{y}^* = \hat{\beta}_0 + \hat{\beta}_1 x^*.$$

Two different prediction questions

We may want to estimate:

1. The **average response** at x^* : $\mathbb{E}[Y \mid X = x^*] = \beta_0 + \beta_1 x^*$.
2. A **single new observation** Y^* : $Y^* = \beta_0 + \beta_1 x^* + \varepsilon^*$.

Both share the same point estimate $\hat{y}^*$, but they involve **fundamentally different uncertainties**.

Confidence Intervals vs. Prediction Intervals

Confidence interval for $\mathbb{E}[Y \mid X = x^*]$ — uncertainty in the *mean response*:

$$\hat{y}^* \pm t_{\alpha/2, n-2} \hat{\sigma} \sqrt{\frac{1}{n} + \frac{(x^* - \bar{x})^2}{S_{xx}}}.$$

Prediction interval for a new observation Y^* — uncertainty in a *single future value*:

$$\hat{y}^* \pm t_{\alpha/2, n-2} \hat{\sigma} \sqrt{1 + \frac{1}{n} + \frac{(x^* - \bar{x})^2}{S_{xx}}}.$$

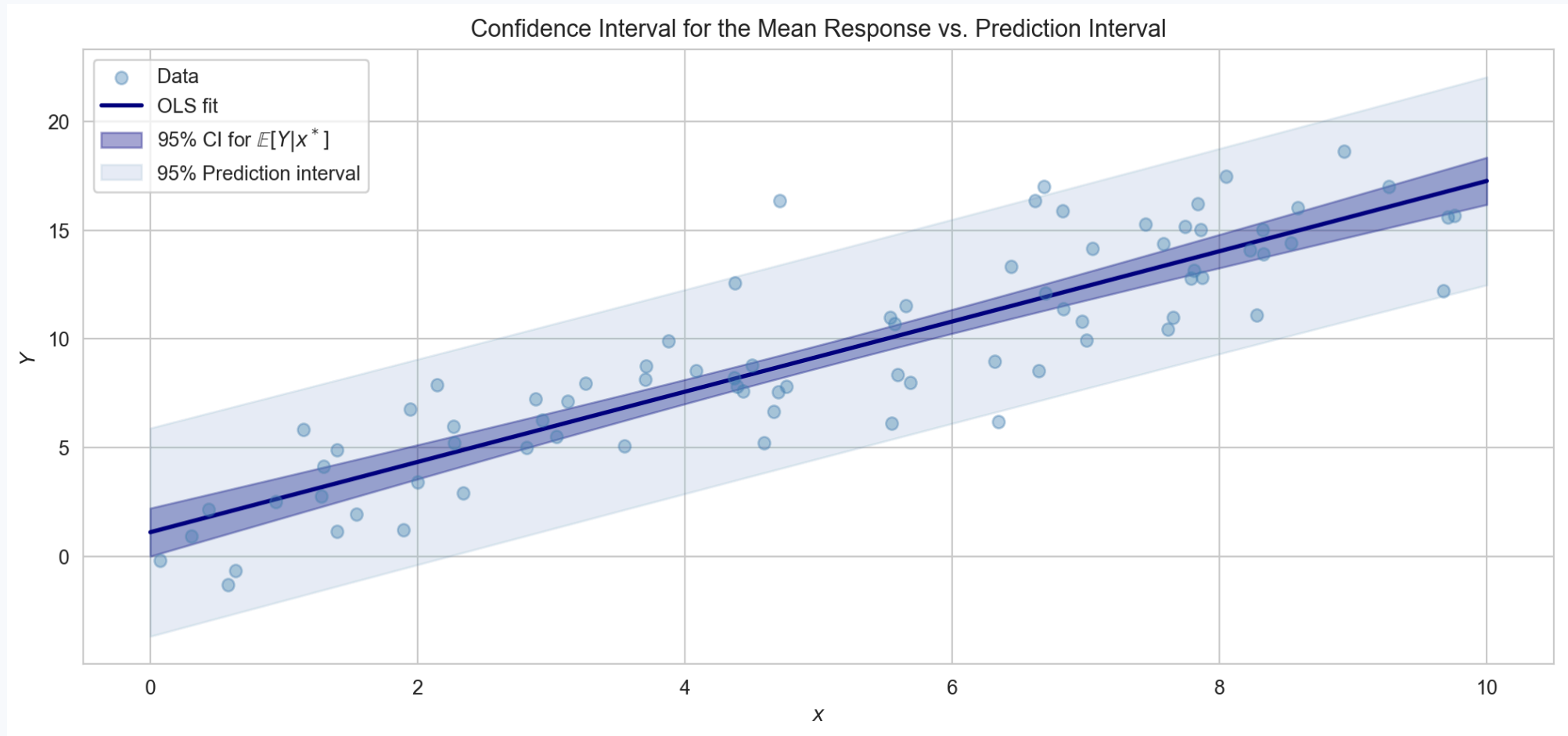
Key differences

- The prediction interval carries an extra $\hat{\sigma}^2$ for the **new irreducible error** ε^* .
- Prediction intervals are **always wider** than confidence intervals.
- Both intervals **widen as x^* moves away from $\bar{x}$** — extrapolation is less certain.

CI vs. PI in Python

```
1 x_pred = np.linspace(0, 10, 300)
2 y_pred = beta0_hat + beta1_hat * x_pred
3 t_crit2 = t_dist.ppf(0.975, df=n - 2)
4
5 se_mean = sigma_hat * np.sqrt(1/n      + (x_pred - x_bar)**2 / S_xx)
6 se_pi   = sigma_hat * np.sqrt(1 + 1/n + (x_pred - x_bar)**2 / S_xx)
7
8 fig, ax = plt.subplots(figsize=(11, 5))
9 ax.scatter(x, y, alpha=0.4, color='steelblue', label='Data', zorder=3)
10 ax.plot(x_pred, y_pred, 'navy', lw=2, label='OLS fit')
11 ax.fill_between(x_pred,
12                y_pred - t_crit2 * se_mean,
13                y_pred + t_crit2 * se_mean,
14                alpha=0.35, color='navy', label='95% CI for  $E[Y|x^*]$ ')
15 ax.fill_between(x_pred,
16                y_pred - t_crit2 * se_pi,
17                y_pred + t_crit2 * se_pi,
18                alpha=0.12, color='steelblue', label='95% Prediction interval')
19 ax.set_xlabel('$x$'); ax.set_ylabel('$Y$')
20 ax.set_title('Confidence Interval for the Mean Response vs. Prediction Interval')
21 ax.legend()
22 plt.tight_layout()
23 plt.show()
```


CI vs. PI in Python



95% confidence band (dark fill) vs. 95% prediction band (light fill). The prediction band is wider because it also accounts for individual observation error.

Summary: The SLR Workflow

Step	Task	Key Formula / Tool
1. Specify	Write down the statistical model	$Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$
2. Estimate	Minimise RSS	$\hat{\beta}_1 = S_{xy}/S_{xx}, \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$
3. Check properties	Verify unbiasedness, invoke Gauss-Markov	BLUE under GM1-GM3
4. Inference	t -tests and CIs for coefficients	$T = \hat{\beta}_1 / \widehat{SE}(\hat{\beta}_1) \sim t_{n-2}$
5. Fit quality	Decompose variation	$R^2 = SSR/SST$
6. Predict	Point estimate + interval	CI for mean vs. PI for new obs.

Conclusion

✓ What we covered

- **Regression** as estimation of $\mathbb{E}[Y \mid X]$ — connecting to Lectures 7 & 8.
- The **SLR statistical model** $Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$ and the four Gauss-Markov assumptions.
- **OLS estimation**: deriving $\hat{\beta}_0$ and $\hat{\beta}_1$ in closed form by minimising RSS.
- **Properties**: unbiasedness, sampling variances, and the Gauss-Markov theorem (BLUE).
- **Inference**: t -tests and confidence intervals for regression coefficients.
- **Goodness of fit**: SST/SSR/SSE decomposition and the R^2 statistic.
- **Prediction**: the distinction between CIs for the mean response and prediction intervals.

What's next?

- Applying SLR end-to-end in Python using a real dataset.
- Fitting a model with `statsmodels` and reading every line of the regression summary.
- Checking the Gauss-Markov assumptions through residual diagnostic plots.

References